
JAVA OOP MASTERY

PHASE 1 - FOUNDATION & OBJECT MODEL

A recall guide built from the concepts we studied: class, object, references, new, state vs behavior, constructors, this, this(), super(), null, garbage collection, initialization order, and static vs instance members.

Goal: understand the execution, not memorize definitions.

Use this document as a re-call manual. Read the diagrams first, then the code, then the “why”.

1. Phase 1 Master Map

```
CLASS
|
+--> OBJECT
|   |
|   +--> STATE (fields)
|   +--> BEHAVIOR (methods)
|
+--> REFERENCE VARIABLES
|
+--> point to objects or null
```

```
new
|
+--> creates object
+--> initialization happens
+--> constructor runs
+--> reference is assigned
```

Core mental model: A class describes what objects look like. An object is the actual instance. A reference variable is a way to reach that object. A constructor initializes the newly created object.

Class	A type/blueprint describing fields and methods.
Object	An actual instance created from a class.
Reference	A variable that refers to an object or is null.
new	Creates a new object and starts its initialization.
State	The current values of an object's fields.
Behavior	The operations exposed by the object's methods.

The most important distinction to preserve: reference != object.

2. Class, Object, State, Behavior

A class is the description. The object is the actual thing created from that description.

```
class Student {  
    String name;  
    int age;  
  
    void study() {  
        System.out.println(name + " is studying");  
    }  
}
```

```
Student (class)
```

```
|  
+-- name  
+-- age  
+-- study()
```

```
Student s1 = new Student();  
|  
+-----> Student object
```

State = data that describes the object right now: name, age. Behavior = what the object can do: study().

One class can create many independent objects. They share the same class definition but hold separate instance state.

```
Student s1 = new Student();  
Student s2 = new Student();
```

```
s1.name = "Rahul";  
s1.age = 22;
```

```
s2.name = "Arun";  
s2.age = 25;
```

```
s1 ---> Object #1  
        name = Rahul  
        age  = 22
```

```
s2 ---> Object #2  
        name = Arun  
        age  = 25
```

The objects are different even though they were created from the same class.

3. Reference Variables - The Big Mental Model

When you write `Student s = new Student();`, the variable `s` is a reference variable. It does not become the object itself.

```
Student s = new Student();
s.name = "Rahul";
```

STACK (mental model) HEAP

```
s -----> Student object
              name = "Rahul"
              age  = 0
```

Use this three-part breakdown:

Student	Reference type
s	Reference variable
new Student()	Creates the object

Two references can point to the same object.

```
Student s1 = new Student();
Student s2 = s1;
```

```
s1.name = "Rahul";
System.out.println(s2.name); // Rahul
```

```
s1 ----+
      |
      +-----> Student object
                  name = Rahul
s2 ----+
```

Key: `Student s2 = s1;` creates no new object. It copies the reference.

Compare with `Student s2 = new Student();` - that creates a second object.

4. Object Identity, ==, and null

For object references, == checks whether two references point to the same object.

```
Student s1 = new Student();
Student s2 = new Student();
System.out.println(s1 == s2); // false
```

```
Student s3 = s1;
System.out.println(s1 == s3); // true
```

s1 ----> Object A <---- s3

s2 ----> Object B

A != B, but s1 and s3 reach the same A

null means “this reference currently refers to no object.” It is not a Student object and not an object at all.

```
Student s = null;
// s refers to no object
```

Important distinction:

```
Student s;
-> local reference exists, but it is not initialized
```

```
Student s = null;
-> reference exists and explicitly points to null
```

```
Student s = new Student();
s = null;
-> object was created, then the reference stopped pointing to it
```

A local variable like Student s; cannot be read before it is initialized; Java reports a compile-time error.

5. What new Student() Means

Do not think of new Student() as a single unexplained magic event. For our mental model, break it into stages.

```
new Student()  
|  
+--> create a new Student object  
|  
+--> initialize its instance state  
|  
+--> run the constructor  
|  
+--> obtain a reference to the object  
|  
+--> assign that reference to the variable
```

Instance fields receive Java's default values unless their declarations or initialization logic provides something else.

```
class Student {  
    String name;  
    int age;  
    boolean active;  
}
```

name	null
age	0
active	false

Do not confuse fields with local variables. A local variable such as int x; inside a method must be assigned before use.

```
void test() {  
    int x;  
    System.out.println(x); // compile-time error  
}
```

6. Constructors - Purpose and Mental Model

A constructor runs as part of object creation and is used to establish the object's initial state.

```
class Student {  
    String name;  
    int age;  
  
    Student() {  
        name = "Unknown";  
        age = 0;  
    }  
}
```

A constructor is identified by the class name and has no return type.

Why explicitly write a constructor? To control how a newly created object begins life. A constructor can execute normal Java code and initialize fields, validate data, or delegate to another constructor.

```
class Student {  
    String name = "Unknown";  
  
    Student() {  
        name = "Rahul";  
        System.out.println("Constructor");  
    }  
}  
  
new Student()  
|  
+--> field initializer: name = "Unknown"  
|  
+--> constructor runs  
    name = "Rahul"  
    print "Constructor"  
|  
+--> object is now initialized
```

Later: `System.out.println(s.name)` -> Rahul

The constructor does not “store Rahul inside itself.” It executes code that changes the newly created object’s state.

7. Default Constructor vs No-Argument Constructor

These terms are related, but not identical.

No-argument constructor	Any constructor with zero parameters.
Default constructor	The no-argument constructor the compiler supplies when you declare no constructors yourself.

```
class Student {  
    String name;  
}
```

```
// You wrote no constructor.  
// Java supplies a default no-argument constructor.
```

Once you write a constructor, Java stops automatically supplying that compiler-generated default constructor.

```
class Student {  
    Student(String name) {  
        this.name = name;  
    }  
  
    String name;  
}
```

```
// new Student(); // compile-time error: no no-arg constructor
```

You may explicitly add one when you need multiple creation paths.

```
Student() { }  
Student(String name) { this.name = name; }
```

8. Constructor Overloading

Constructor overloading gives different ways to initialize the same class.

```
class Student {
    String name;
    int age;

    Student() {
        name = "Unknown";
        age = 0;
    }

    Student(String name) {
        this.name = name;
        age = 0;
    }

    Student(String name, int age) {
        this.name = name;
        this.age = age;
    }
}

new Student()
    -> Student()

new Student("Rahul")
    -> Student(String)

new Student("Rahul", 22)
    -> Student(String, int)
```

The constructor name stays the same. What changes is the parameter list. Java chooses the matching constructor using the arguments supplied in the call.

Business value: different creation paths for objects that may need different amounts of initial information.

9. this - Current Object

The keyword `this` refers to the current object whose constructor or instance method is executing.

```
class Student {  
    String name;  
  
    Student(String name) {  
        this.name = name;  
    }  
}
```

Student	object field		constructor parameter
	<code>this.name</code>	<code><---</code>	<code>name</code>
	<code>+----- assignment-+</code>		

Without `this`, when a constructor parameter has the same name as the field, both names resolve to the parameter in that scope. That is why `this.name = name;` is so common.

```
Student(String name) {  
    name = name;           // does not update the field  
  
    this.name = name;      // field <- parameter  
}
```

Three ideas to keep separate: `this.name` = current object's field; `this(...)` = another constructor in the same class; `super(...)` = a parent constructor.

10. this() - Constructor Chaining

this(...) invokes another constructor in the same class. It is a constructor invocation, not a normal method.

```
class Student {  
  
    Student() {  
        this("Unknown", 0);  
    }  
  
    Student(String name) {  
        this(name, 0);  
    }  
  
    Student(String name, int age) {  
        this.name = name;  
        this.age = age;  
    }  
  
    String name;  
    int age;  
}  
  
new Student()  
    |  
    v  
Student()  
    | this("Unknown", 0)  
    v  
Student(String, int)  
    |  
    +--> actual initialization
```

Why use it? To avoid duplicated initialization logic and keep one source of truth for object rules and validation.

```
Student() {  
    this("Unknown", 0);  
    System.out.println("No-arg constructor");  
}
```

Critical rule: this(...) must be the first statement in a constructor.

```
Student() {  
    this("Unknown"); // must come first  
    System.out.println("Hello");  
}
```

Do not say “this() goes to the default constructor.” Say: this() goes to another constructor in the same class. It can target a no-arg or parameterized constructor.

11. super() - Parent Constructor

super(...) invokes a constructor in the parent class.

```
class Animal {  
    Animal() {  
        System.out.println("Animal");  
    }  
}
```

```
class Dog extends Animal {  
    Dog() {  
        super();  
        System.out.println("Dog");  
    }  
}
```

```
new Dog()  
  |  
  v  
Dog()  
  |  
  +--> super()  
  |      |  
  |      v  
  |    Animal()  
  |  
  +--> return to Dog()  
        |  
        +--> Dog constructor body
```

If you do not explicitly write a constructor call, Java implicitly inserts a no-argument `super()` call when the parent has an accessible no-argument constructor.

Thus, in the simple case, parent construction happens before child construction.

Key difference: `this(...)` = same class. `super(...)` = parent class.

12. Constructor Execution Order - Trace It

```
class Account {  
    String type = "Unknown";  
  
    Account() {  
        this("Savings");  
        System.out.println("A");  
    }  
  
    Account(String type) {  
        this.type = type;  
        System.out.println("B");  
    }  
}
```

```
Account a = new Account();  
System.out.println(a.type);
```

```
new Account()  
  |  
  v  
Account()  
  |  
  +--> this("Savings")  
        |  
        v  
    Account(String)  
        |  
        +--> print B  
        |  
        +--> return  
    |  
    +--> print A  
    |  
    +--> constructor complete  
    |  
    v  
main continues -> print Savings
```

Exact output:

```
B  
A  
Savings
```

The main method's `System.out.println(a.type)` executes only after the constructor call has completely finished.

13. Instance Field Initializers and Initializer Blocks

```
class Student {
    String name = "Unknown";

    {
        System.out.println("Initializer");
    }

    Student() {
        System.out.println("Constructor");
    }
}
```

For a class without inheritance, a useful mental model is:

```
object creation
|
+--> instance field initialization
|
+--> instance initializer blocks (source order)
|
+--> constructor body
```

Multiple initializer blocks execute in the order they appear in the source.

```
class Student {
    { System.out.println("Block 1"); }
    { System.out.println("Block 2"); }

    Student() {
        System.out.println("Constructor");
    }
}
```

Output:
Block 1
Block 2
Constructor

Initializer blocks are useful to understand initialization, but everyday Java code more commonly uses field initializers and constructors directly.

14. Parent First, Child Second

```
class Animal {
    String type = "Animal";

    Animal() {
        System.out.println("Animal constructor");
    }
}

class Dog extends Animal {
    String name = "Dog";

    Dog() {
        System.out.println("Dog constructor");
    }
}
```

When new Dog() executes, the parent portion is initialized before the child portion. A simplified mental model is:

```
new Dog()
|
+--> parent field initialization
+--> parent initializer blocks
+--> parent constructor
|
+--> child field initialization
+--> child initializer blocks
+--> child constructor
```

This is why constructor output is parent first, then child. It also explains the invisible super() call in the simple no-argument case.

15. Garbage Collection - Reachability

Java uses automatic memory management. You normally do not manually free an object.

```
Student s = new Student();  
s = null;
```

Before:
s ----> Student object

After s = null:
s ----> null

Student object
(no reachable reference from s)

The object does not become null. The reference becomes null. The object becomes unreachable (from that reference path) and therefore eligible for garbage collection.

Eligible does not mean immediately deleted. The JVM decides when memory is actually reclaimed.

```
System.gc();
```

System.gc() is only a request/suggestion to the JVM, not a guarantee that collection happens immediately.

```
Student a = new Student();  
Student b = new Student();  
a = b;
```

Before:
a ---> Object A
b ---> Object B

After:
a ---+
 +----> Object B
b ---+

Object A has no reference from a/b and is eligible for GC.

16. static vs Instance Members

This is one of the most important Phase 1 distinctions.

```
class Student {  
    String name;  
    static String college = "ABC";  
}
```

```
Student class  
|  
+--> static college = "ABC" (shared)
```

```
Object #1  
+--> name = Rahul
```

```
Object #2  
+--> name = Arun
```

Instance field = belongs to each object. Static field = class-level/shared state.

```
Student s1 = new Student();  
Student s2 = new Student();
```

```
s1.name = "Rahul";  
s2.name = "Arun";
```

```
Student.college = "XYZ";
```

Both objects observe the same static college value, while each object has its own name.

17. static Methods - Why They Differ

```
class Student {
    String name;

    void printName() {
        System.out.println(name);
    }

    static void collegeInfo() {
        System.out.println("ABC College");
    }
}

Student s = new Student();
s.name = "Rahul";
s.printName();

Student.collegeInfo();
```

The instance method has a particular object context: `s.printName()`. The static method is invoked through the class and does not have one particular `Student` object as its context.

Therefore, a static method cannot directly access an instance field like `name` without being given an object/reference to use.

```
class Student {
    String name;

    static void printName() {
        // System.out.println(name); // compile-time error
    }
}
```

Conceptual question to ask: “Which object’s state would this static code use?” If there is no specific object, direct instance access does not make sense.

18. The Counter Example - Instance vs Static

```
class Counter {
    int x = 0;
    static int y = 0;

    Counter() {
        x++;
        y++;
    }
}

Counter a = new Counter();
Counter b = new Counter();
Counter c = new Counter();

System.out.println(a.x);
System.out.println(b.x);
System.out.println(c.x);
System.out.println(Counter.y);

Object A: x = 1
Object B: x = 1
Object C: x = 1

Counter class: y = 3
```

Exact output:

```
1
1
1
3
```

Each object starts its own x at 0, so each constructor makes it 1. All objects share the one static y, so three constructor calls make it 3.

19. The Syntax You Must Be Able to Read Instantly

<code>this.name</code>	Current object's instance field named <code>name</code> .
<code>this(...)</code>	Call another constructor in the same class.
<code>super(...)</code>	Call a constructor in the parent class.
<code>obj.field</code>	Access an instance field through a reference.
<code>obj.method()</code>	Invoke an instance method on that object.
<code>ClassName.staticField</code>	Access shared class-level state.
<code>ClassName.staticMethod()</code>	Invoke class-level behavior that does not require a particular object.
<code>==</code> (object references)	Checks whether two references point to the same object.
<code>null</code>	Reference refers to no object.

Three phrases to permanently anchor:

"Copying a reference is not copying an object."

"`this()` means same class; `super()` means parent class."

"Static is class-level/shared; instance is object-level/individual."

20. Phase 1 Checkpoint Answers

Challenge 1: Two objects, three references. a and c point to the same object. b points to another object.

```
Arun  
null  
Arun
```

Challenge 2: this("Savings") delegates from the no-arg constructor to the parameterized constructor. Therefore B prints first, control returns, then A prints, and only afterward does main print the field value.

```
B  
A  
Savings
```

Challenge 3: Each object has its own x, while y is shared by the class.

```
1  
1  
1  
3
```

These were not "just answers." The important part was your reference/object reasoning and your constructor execution tracing.

21. Common Phase 1 Traps

"s is the object"	Better: s is a reference to the object.
"s2 = s1 creates another object"	No. It copies the reference; use new to create another object.
"null is an object"	No. null means the reference points to no object.
"the object becomes null"	No. the reference becomes null; the object may become unreachable.
"this() means default constructor"	No. it means another constructor in the same class.
"super() is just inherited behavior"	It is a constructor invocation to the parent class.
"static field belongs to every object separately"	No. it is class-level/shared.
"constructor stores values inside itself"	Constructor code initializes or changes the object being created.
"no-arg = default"	Every default constructor is no-arg, but not every no-arg constructor is compiler-generated default.

When confused, redraw the references. Most Phase 1 confusion disappears once you separate class, object, reference, and shared class state.

22. One-Page Recall Checklist

Before moving to Phase 2, you should be able to answer these instantly:

- What is the difference between a class and an object?
- What is a reference variable?
- How many objects are created by repeated new expressions?
- What happens when two references point to the same object?
- What does null mean?
- What does new do conceptually?
- Why do instance fields get default values?
- What is a constructor and why is it useful?
- What is the difference between a default constructor and a no-arg constructor?
- What is constructor overloading?
- Why use this(...)?
- Why must this(...) be first?
- What does this.name mean?
- What does super(...) mean?
- Why does parent construction happen before child construction?
- When does an object become eligible for garbage collection?
- What is the difference between static and instance state?
- Why can a static method not directly use instance state?

If you can explain each one in your own words and trace the code without guessing, Phase 1 is doing its job.

23. Final Mental Map - Keep This Forever

```
class Student { ... }
    |
    | describes structure + behavior
    v
new Student()
    |
    +--> object exists
    |
    +--> fields initialize
    |
    +--> constructors execute
    |
    |     |
    |     +--> this(...) = same class constructor
    |     +--> super(...) = parent constructor
    |
    v
reference variable receives object reference
    |
    +--> obj.field / obj.method()
    |
    +--> can be reassigned to null
    |
    +--> shared class state lives in static members
```

Think:
CLASS = description
OBJECT = actual instance
REFERENCE = way to reach object
STATE = current field values
BEHAVIOR = methods
CONSTRUCTOR = initialization code
this = current object
this() = another constructor, same class
super() = parent constructor
static = class-level/shared

This is the foundation we will reuse when we enter encapsulation, inheritance, polymorphism, abstraction, generics, and the rest of Java OOP.

PHASE 1 COMPLETE